

PROJECT REPORT

TANSU Niraloy: A Centralized Property Rental and Sale Management Platform with a Verified Intermediary Workflow

Course: CSE 2118: Advanced Object-Oriented Programming

Department: B.Sc. in Computer Science and Engineering (CSE)

Student Name	Nurul Islam Dipu. MD.Arafath Hasan Khan
Student Id	0112330628. 0112410277.
Section	H
Submitted To	MD. FOZLUL Haque
Date of submission	30/9/2026
Github repository Link	https://github.com/nidipu11/TANSU-Niraloy.git

Abstract

Finding, leasing and buying residential property in a fast-growing city such as Dhaka is still dominated by informal brokers, unverified classified listings and direct, undocumented dealings between tenants and landlords. This leads to ghost listings, disputes over security deposits, delayed rent disbursement to owners and the absence of any neutral party who can mediate. This project presents TANSU Niralay, a RESTful web backend built with Spring Boot 3.2.3 that acts as a verified intermediary between property owners and tenants. The platform enforces one golden rule: owners and tenants never transact directly; every listing verification, visit, lease, payment and complaint passes through the TANSU administrative desk.

The backend uses Spring Data JPA (Hibernate) with MySQL persistence, Jakarta Bean Validation for request validation, constructor-based dependency injection, and an interface-driven Controller, Service and Repository architecture. Advanced Object-Oriented Programming concepts and design patterns, namely abstraction and polymorphism through service interfaces, the Builder pattern, the Static Factory Method pattern, the Data Transfer Object (DTO) pattern and the Repository pattern, are applied throughout the code base. The system supports three roles (TENANT, OWNER and ADMIN) and exposes more than 50 REST endpoint mappings that cover property vetting, rental, purchase and visit requests, automatic rent-due generation, payment processing, an 80/20 owner settlement and escrow engine, support tickets, recovery notices and operating-expense accounting. A responsive HTML5, CSS3 and Vanilla JavaScript client consumes the API. The backend was verified through API testing with Postman, demonstrating correct data persistence, validation and business-rule behaviour across complete multi-role workflows.

1. Introduction

1.1 Background and Problem Statement

Residential rental and sale transactions in Bangladesh are often arranged through informal channels. Tenants face fake or outdated listings, hidden charges and uncertainty about whether the person they are dealing with really owns the property. Owners face late payments, manual rent collection and no formal payment trail. When a maintenance issue or a contractual disagreement arises, there is no accredited third party to resolve it.

To address these problems, TANSU Niralay delivers a unified RESTful backend service. A REST API provides a standardized, decoupled architecture that allows the web front-end (and, in future, a mobile application) to interact with the same backend resources. Spring Boot was selected as the core framework because of its auto-configuration, embedded Tomcat server, built-in dependency injection container and seamless data access through Spring Data JPA.

The business flow implemented by the system is shown below.

```

[ PROPERTY OWNER ]
  | submits listing (title, location, price, photos, details)
  v
[ TANSU NIRALOY ADMIN / SYSTEM ] -- vets the listing (PENDING -> VERIFIED)
  | publishes verified listing to the public catalog
  v
[ PROSPECTIVE TENANT ] -- sends Rent / Buy / Visit request through the TANSU desk
  |
  v
[ TANSU NIRALOY ADMIN ] -- approves request, rent due is generated automatically
  |
  v
[ PAYMENT + SETTLEMENT ] -- 20% platform fee kept, 80% credited to the owner

```

1.2 Objectives

- Implement CRUD operations for properties and the complete request, billing, payment and support workflows through structured REST endpoints.
- Provide secure registration and sign-in for three segregated roles: TENANT, OWNER and ADMIN, with role-specific API groups (/api/tenant, /api/owner, /api/admin).
- Enforce an administrative vetting workflow so that a new listing stays in PENDING status until an admin verifies it.
- Automate monthly rent-due generation when a rental request is approved, and update the property availability (AVAILABLE_FOR_RENT to RENTED).
- Implement an intermediary settlement engine that retains a 20% platform fee, credits 80% to the owner, and supports escrow holding with scheduled and manual release.
- Apply Object-Oriented design principles and patterns (interfaces, dependency injection, Builder, Static Factory, DTO and Repository) to keep the code decoupled and testable.
- Persist all domain entities in a relational database using Spring Data JPA and Hibernate ORM.
- Return precise HTTP responses (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict) wrapped in a uniform JSON envelope.

1.3 Scope

In Scope:

- **Authentication and User Management:** registration, sign-in, role handling and a login audit log.
- **Property Management:** owner listing submission, admin verification queue (approve / reject), public catalog search and filtering by location, type, purpose, price and bedrooms.
- **Request Pipeline:** rent, buy and escorted-visit requests, admin approval, rejection and escort-officer assignment.
- **Billing and Payments:** tenant dues, payment checkout processing (card, bKash, Nagad, Rocket and net-banking style methods), collections ledger and recovery notices for overdue dues.

- **Owner Settlement and Escrow:** 80/20 settlement records, owner withdrawal requests, admin approval of disbursements, escrow hold and release.
- **Support and Accounting:** tenant support tickets, operating-expense records and platform finance metrics.

Out of Scope:

Integration with real production payment gateways (payments are processed as a secure simulation), live SMS or e-mail delivery, GPS tracking of field officers, and container orchestration or microservice deployment. The system is deliberately a layered monolith.

2. Tools and Technologies

Category	Tool / Version	Why it was used
Language	Java 17	Long-term-support release with modern language features (streams, text blocks, records) and strong OOP support.
Framework	Spring Boot 3.2.3	Rapid backend development with auto-configuration, embedded Tomcat server and a dependency injection container.
Web Layer	Spring Web MVC (spring-boot-starter-web)	Exposes REST controllers, request routing, JSON serialization with Jackson and HTTP parameter binding.
Persistence	Spring Data JPA / Hibernate ORM	Object-relational mapping, derived query methods, custom JPQL queries and <code>@Transactional</code> boundaries without hand-written JDBC code.
Database	MySQL (XAMPP, port 3306)	Relational database with ACID transactions; the database is created automatically on first start.
Validation	Jakarta Bean Validation (spring-boot-starter-validation)	Declarative validation of incoming DTOs using <code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@Positive</code> and <code>@NotNull</code> .
Utilities	Project Lombok	Removes boilerplate (getters, setters, constructors) and generates builders with <code>@Builder</code> .
Build Tool	Apache Maven	Dependency management, build automation and the <code>spring-boot:run</code> goal.
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)	Framework-free responsive client that calls the REST API using the Fetch API.
IDE & API Testing	IntelliJ IDEA / VS Code / Postman	Development environment, Live Server for the front-end and endpoint testing suite.
Version Control	Git and GitHub	Source code management and project submission.

3. Project Setup

The backend was initialized using Spring Initializr (start.spring.io) with the following settings, and the front-end client was developed as a separate folder of static pages.

Initializr Field	Value
Project / Language	Maven / Java 17
Spring Boot Version	3.2.3
Group / Artifact	com.tansu / tansu-backend
Package Name	com.tansu.niraloy
Packaging	Jar
Dependencies	Spring Web, Spring Data JPA, Validation, MySQL Driver, Lombok, Spring Boot Test

Backend Package Structure

```

tansu-backend
├── pom.xml
├── src/main
│   ├── java/com/tansu/niraloy
│   │   ├── TansuNiraloyApplication.java
│   │   ├── config
│   │   │   ├── CorsConfig.java
│   │   │   └── DatabaseSeeder.java
│   │   ├── controller
│   │   │   ├── AdminController.java
│   │   │   ├── AuthController.java
│   │   │   ├── OwnerController.java
│   │   │   ├── PropertyController.java
│   │   │   ├── PropertyRequestController.java
│   │   │   └── TenantController.java
│   │   ├── dto
│   │   │   ├── ApiResponse.java
│   │   │   ├── AuthDto.java
│   │   │   ├── FinanceDto.java
│   │   │   ├── PaymentDto.java
│   │   │   ├── PropertyDto.java
│   │   │   ├── PropertyRequestDto.java
│   │   │   └── SupportTicketDto.java
│   │   ├── exception
│   │   │   ├── EmailAlreadyExistsException.java
│   │   │   ├── InvalidCredentialsException.java
│   │   │   └── UserNotFoundException.java
│   │   ├── model
│   │   │   ├── OperatingExpense.java
│   │   │   ├── OwnerSettlement.java
│   │   │   ├── Payment.java
│   │   │   ├── Property.java
│   │   │   ├── PropertyRequest.java
│   │   │   ├── RecoveryNotice.java
│   │   │   ├── Role.java
│   │   │   ├── SupportTicket.java
│   │   │   ├── TenantCollection.java
│   │   │   ├── TenantDue.java
│   │   │   ├── User.java
│   │   │   └── UserLoginLog.java
│   │   ├── repository (11 Spring Data JPA interfaces, one per entity)
│   │   └── service
│   │       ├── AuthService / FinanceService / PaymentService
│   │       ├── PropertyService / PropertyRequestService / SupportTicketService
│   │       └── impl (one *ServiceImpl class for each interface above)
│   └── resources
│       └── application.properties

```

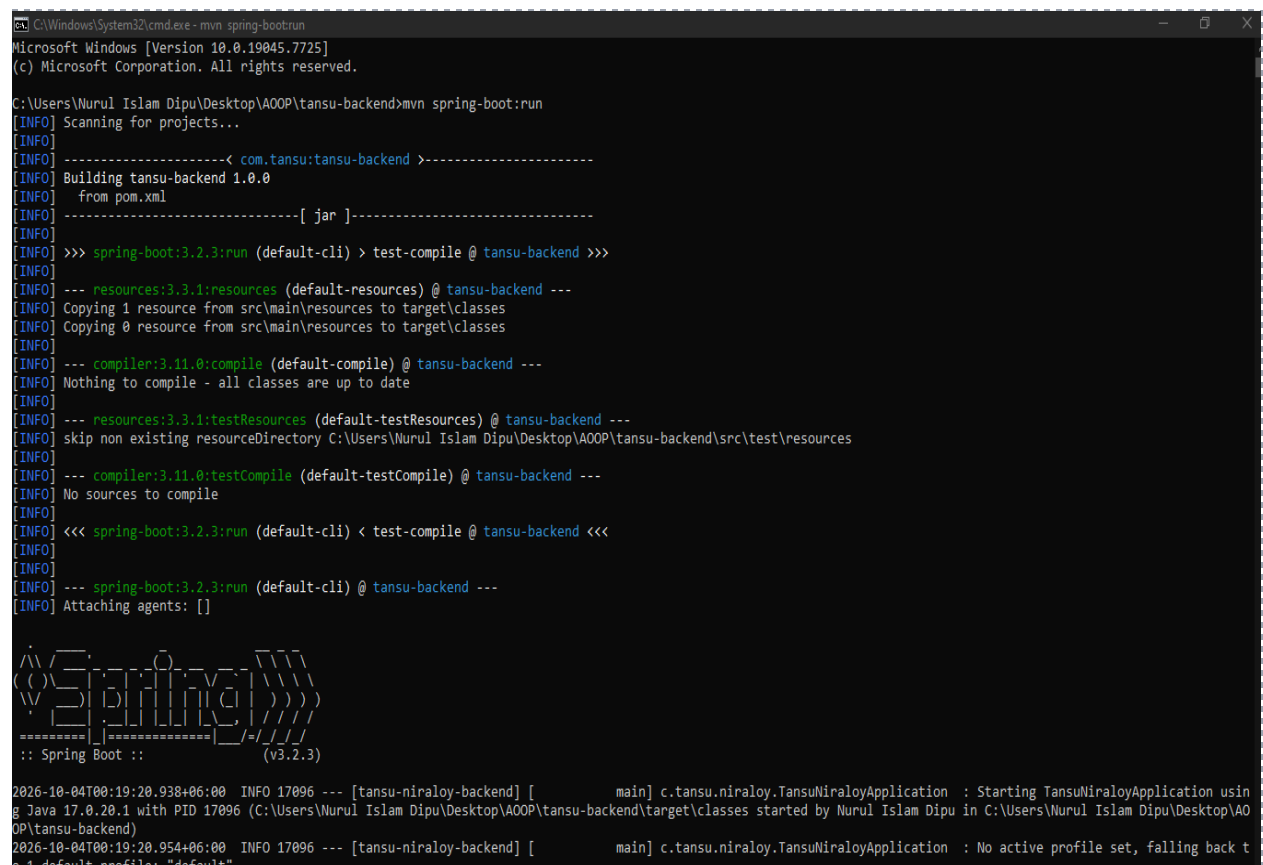
Frontend Structure

The client (folder TANSU-NIRALLOY) contains more than 30 HTML pages grouped by role (public catalog, tenant dashboard, owner dashboard and an admin console with verification, request, finance, transaction, support and report pages), three CSS files (style.css, components.css, responsive.css) and ten JavaScript modules (api.js, auth.js, properties.js, tenant.js, owner.js, admin.js, payments.js, validation.js, components.js, app.js). The base URL of the backend is configured once in js/api.js as `http://localhost:8080`.

How to Run the Project

1. Start the MySQL service (for example from the XAMPP control panel). No manual schema creation is needed because the JDBC URL contains `createDatabaseIfNotExist=true`.
2. Open a terminal in the `tansu-backend` folder and run the backend (shown in Figure 1). The API starts on `http://localhost:8080`.
3. Open the TANSU-NIRALLOY folder in VS Code and launch `index.html` with Live Server (or open it directly in a browser).

```
cd tansu-backend
mvn spring-boot:run
```



```
C:\Windows\System32\cmd.exe - mvn spring-boot:run
Microsoft Windows [Version 10.0.19045.7725]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nurul Islam Dipu\Desktop\AOOP\tansu-backend>mvn spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.tansu:tansu-backend >-----
[INFO] Building tansu-backend 1.0.0
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot:3.2.3:run (default-cli) > test-compile @ tansu-backend >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ tansu-backend ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ tansu-backend ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ tansu-backend ---
[INFO] skip non existing resourceDirectory C:\Users\Nurul Islam Dipu\Desktop\AOOP\tansu-backend\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ tansu-backend ---
[INFO] No sources to compile
[INFO]
[INFO] <<< spring-boot:3.2.3:run (default-cli) < test-compile @ tansu-backend <<<
[INFO]
[INFO] --- spring-boot:3.2.3:run (default-cli) @ tansu-backend ---
[INFO] Attaching agents: []

  ____ _
 / ___ \ | |
/ /___\ \| |
 \___)___)___|_|
  Spring Boot :: (v3.2.3)

2026-10-04T00:19:20.938+06:00 INFO 17096 --- [tansu-niraloy-backend] [main] c.tansu.niraloy.TansuNiraloyApplication : Starting TansuNiraloyApplication using Java 17.0.20.1 with PID 17096 (C:\Users\Nurul Islam Dipu\Desktop\AOOP\tansu-backend\target\classes started by Nurul Islam Dipu in C:\Users\Nurul Islam Dipu\Desktop\AOOP\tansu-backend)
2026-10-04T00:19:20.954+06:00 INFO 17096 --- [tansu-niraloy-backend] [main] c.tansu.niraloy.TansuNiraloyApplication : No active profile set, falling back to the default profile: "default"
```

Figure 1: TANSU Niraloy Spring Boot backend starting successfully on port 8080.

4. System Architecture

TANSU Niraloy follows a clean, layered architecture that enforces a strict separation of concerns between the presentation, business logic and persistence layers. Each layer only talks to the layer directly below it, and every dependency between layers is expressed through an interface and injected by Spring.

```
Client (Web Frontend / Postman)
  | HTTP Requests (JSON)
  v
Controller Layer (@RestController)
  - Maps REST endpoints, validates DTOs with @Valid, returns ResponseEntity
  |
  v
Service Layer (@Service, @Transactional)
  - Business rules: verification, lease approval, billing, 80/20 settlement
  |
  v
Repository Layer (JpaRepository)
  - Derived queries, JPQL queries and ORM mappings
  |
  v
Database (MySQL)
  - users, properties, property_requests, tenant_dues, payments, ...
```

Request Lifecycle Example (POST /api/requests followed by admin approval):

1. The tenant portal submits an HTTP POST request with a JSON body (PropertyRequestDto) to /api/requests.
2. PropertyRequestController receives the request. The @Valid annotation triggers Bean Validation; a missing propertyId, tenantId or requestType produces 400 Bad Request.
3. The controller delegates to the PropertyRequestService interface. Spring injects PropertyRequestServiceImpl at runtime.
4. The service completes missing tenant name or e-mail from UserRepository, builds a PropertyRequest with status PENDING using the Builder and saves it through PropertyRequestRepository.
5. The @PrePersist callback generates a business identifier (for example REQ-48213) and the request date; Hibernate issues the SQL INSERT against MySQL.
6. The saved entity is wrapped in the uniform ApiResponse envelope and returned with HTTP 200 OK.
7. Later, the admin calls PUT /api/requests/{id}/approve. Inside a single @Transactional method the request becomes APPROVED, a TenantDue is created for the first rent plus utility charge, and the property availability changes to RENTED.

Payment and Settlement Flow:

When a tenant pays a due through POST /api/tenant/payments, PaymentServiceImpl records the Payment, marks the matching TenantDue as PAID, writes a TenantCollection ledger entry and creates an OwnerSettlement that credits 80% of the amount to the owner and

records a 20% platform commission. The admin later approves the settlement, which moves it to `SETTLED`. A separate escrow path holds a payment as `HELD_IN_ESCROW` for 24 hours and releases it automatically through an hourly `@Scheduled` job, or immediately when an admin approves it.

5. Implementation

5.1 Advanced OOP Concepts and Design Patterns

To demonstrate Advanced Object-Oriented Programming (AOOP) principles, the project applies several well-known object-oriented concepts and design patterns instead of placing all logic in controllers. The table below summarizes where each concept is used and what benefit it provides.

Concept / Pattern	Where it is used	Benefit
Abstraction and Polymorphism (interface-based design)	<code>PropertyService</code> , <code>PropertyRequestService</code> , <code>PaymentService</code> , <code>FinanceService</code> , <code>AuthService</code> and <code>SupportTicketService</code> are interfaces; each has a <code>*ServiceImpl</code> class.	Controllers depend on abstractions, not concrete classes. An implementation can be replaced or mocked without changing any controller.
Dependency Injection / Inversion of Control	Constructor injection with final fields generated by Lombok <code>@RequiredArgsConstructor</code> in every controller and service.	Loose coupling, immutability of dependencies and easy unit testing with mock repositories.
Builder Pattern	<code>@Builder</code> on all entities and on <code>ApiResponse</code> ; <code>@Builder.Default</code> supplies safe default values such as <code>status = "PENDING"</code> .	Readable object construction without long constructors and fewer mistakes with optional fields.
Static Factory Method	<code>ApiResponse.ok(...)</code> and <code>ApiResponse.error(...)</code> ; <code>Role.fromString(...)</code> annotated with <code>@JsonCreator</code> .	Named, intention-revealing creation methods that centralize object creation and default handling.
DTO (Data Transfer Object) Pattern	<code>AuthDto</code> , <code>PropertyDto</code> , <code>PropertyRequestDto</code> , <code>PaymentDto</code> , <code>SupportTicketDto</code> , <code>FinanceDto</code> .	Separates the API contract and validation rules from the persistence model.
Repository Pattern	Eleven <code>JpaRepository</code> interfaces such as <code>PropertyRepository</code> and <code>TenantDueRepository</code> .	Hides persistence details behind collection-like interfaces; Spring generates the implementations.
Encapsulation and Lifecycle Callbacks	Private entity fields with Lombok accessors; <code>@PrePersist</code> , <code>@PreUpdate</code> and <code>@PostLoad</code> hooks in entities.	Entities keep their own invariants (generated IDs, timestamps, consistent status values).
Custom Exception Hierarchy	<code>UserNotFoundException</code> , <code>InvalidCredentialsException</code> , <code>EmailAlreadyExistsException</code> .	Domain errors are mapped to precise HTTP status codes (404, 401 and 409).
Singleton Scope	Controllers, services and repositories are Spring-managed singleton beans.	One shared, thread-safe instance per component and centralized lifecycle management.

Service Interface (abstraction):

```
public interface PropertyService {
    List<Property> searchProperties(String location, String propertyType,
                                   String purpose, Double maxPrice,
                                   Integer bedrooms);

    Property getPropertyById(String propertyId);
    Property createProperty(PropertyDto.CreatePropertyRequest request);
    Property updateProperty(String propertyId,
                            PropertyDto.UpdatePropertyRequest request);
    void deleteProperty(String propertyId);
    List<Property> getPropertiesByOwner(String ownerId);
    List<Property> getPendingProperties();
    Property approveProperty(String propertyId);
    Property rejectProperty(String propertyId);
}
```

Static Factory Methods and Builder (uniform response envelope):

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ApiResponse<T> {
    private boolean success;
    private String message;
    private Integer count;
    private T data;

    public static <T> ApiResponse<T> ok(String message, T data) {
        return ApiResponse.<T>builder()
            .success(true).message(message).data(data).build();
    }

    public static <T> ApiResponse<T> error(String message) {
        return ApiResponse.<T>builder()
            .success(false).message(message).build();
    }
}
```

Static Factory Method on an enum (safe role parsing):

```
public enum Role {
    GUEST, TENANT, OWNER, ADMIN;

    @JsonCreator
    public static Role fromString(String value) {
        if (value == null || value.trim().isEmpty()) {
            return TENANT;
        }
        try {
            return Role.valueOf(value.trim().toUpperCase());
        } catch (IllegalArgumentException e) {
            return TENANT;
        }
    }
}
```

5.2 Domain Entity: Property

`Property` represents a listed apartment, family house, bachelor flat or flat for sale. It is the central entity of the catalog and of the verification workflow.

Field	Java Type	Annotations	Description
id	Long	@Id, @GeneratedValue(IDENTITY)	Auto-incremented primary key
propertyId	String	@Column(unique, nullable=false)	Business identifier such as PROP-GLS-01
ownerId	String	@Column(nullable=false)	userId of the owner who submitted the listing
title	String	@Column(nullable=false, length=200)	Listing headline
location	String	@Column(nullable=false)	Dhaka zone (Gulshan, Banani, Mirpur, Bashundhara, ...)
propertyType	String	@Column(name="property_type")	FAMILY_HOUSE, BACHELOR_HOUSE, FLAT_SALE or PREMIUM_FAMILY_HOUSING
purpose	String	@Column(nullable=false)	RENT or SALE
price	Double	@Column(nullable=false)	Monthly rent or sale price in BDT
area / bedrooms / bathrooms	Integer	@Column	Size in square feet and room counts
amenities	List<String>	@ElementCollection	Stored in table property_amenities
images	List<String>	@ElementCollection	Image URLs stored in table property_images
description	String	@Column(columnDefinition="TEXT")	Detailed description
status	String	@Column, @Builder.Default	PENDING, APPROVED or REJECTED
verificationStatus	String	@Column(name="verification_status")	PENDING_VERIFICATION, VERIFIED or REJECTED
availabilityStatus	String	@Column(name="availability_status")	AVAILABLE_FOR_RENT, AVAILABLE_FOR_SALE, RENTED, SOLD ...
submissionDate / createdAt / updatedAt	LocalDate	@Column	Audit dates maintained by lifecycle callbacks

```
@Entity
@Table(name = "properties")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Property {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false, unique = true, length = 50)
private String propertyId;          // e.g. PROP-GLS-01

@Column(nullable = false, length = 50)
private String ownerId;             // reference to the owner's userId

@Column(nullable = false, length = 200)
private String title;

@Column(nullable = false)
private Double price;

@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "property_amenities",
                 joinColumns = @JoinColumn(name = "property_id"))
@Column(name = "amenity")
private List<String> amenities;

@Column(name = "status", length = 30)
@Builder.Default
private String status = "APPROVED"; // PENDING, APPROVED, REJECTED

// ... location, propertyType, purpose, area, bedrooms, bathrooms,
//      description, images, verificationStatus, availabilityStatus, dates

@PrePersist
protected void onCreate() {
    if (this.createdAt == null) this.createdAt = LocalDate.now();
    if (this.propertyId == null || this.propertyId.isEmpty()) {
        this.propertyId = "PROP-" + (1000 + (System.currentTimeMillis() % 90000));
    }
}
}

```

In my own words: `@Entity` tells Hibernate that the class is mapped to a table, and `@Table` names it `properties`. The numeric `id` is the technical primary key, while `propertyId` is a human-readable business key that is used in URLs and in other tables, so internal sequence numbers are never exposed. The default status `APPROVED` is used only for seeded demo data; when an owner creates a listing, the service explicitly sets it to `PENDING` so that no new property can bypass the admin verification desk.

5.3 Repository Layer

Spring Data JPA generates the repository implementations at start-up from the declared interface methods. `PropertyRepository` combines derived query methods with a custom JPQL query that implements the multi-criteria catalog search (only verified, non-inactive listings are returned, and every filter is optional).

```

@Repository
public interface PropertyRepository extends JpaRepository<Property, Long> {

    Optional<Property> findByPropertyId(String propertyId);
    List<Property> findByOwnerIdOrderByCreatedAtDesc(String ownerId);
    List<Property> findByVerificationStatus(String verificationStatus);
    List<Property> findByStatusOrderByCreatedAtDesc(String status);

    @Query("SELECT p FROM Property p WHERE " +
        "(p.status = 'APPROVED' OR p.verificationStatus = 'VERIFIED') " +
        "AND (p.availabilityStatus IS NULL " +
        "OR p.availabilityStatus != 'INACTIVE') " +
        "AND (:location IS NULL OR :location = 'ALL' " +
        "OR LOWER(p.location) = LOWER(:location)) " +
        "AND (:propertyType IS NULL OR :propertyType = 'ALL' " +
        "OR p.propertyType = :propertyType) " +
        "AND (:purpose IS NULL OR :purpose = 'ALL' " +
        "OR p.purpose = :purpose) " +
        "AND (:maxPrice IS NULL OR p.price <= :maxPrice) " +
        "AND (:bedrooms IS NULL OR p.bedrooms >= :bedrooms) " +
        "ORDER BY p.createdAt DESC")
    List<Property> searchProperties(
        @Param("location") String location,
        @Param("propertyType") String propertyType,
        @Param("purpose") String purpose,
        @Param("maxPrice") Double maxPrice,
        @Param("bedrooms") Integer bedrooms);
}

```

By extending `JpaRepository<Property, Long>`, the interface inherits `save`, `findById`, `findAll`, `delete` and other CRUD methods with no code. Methods such as `findByPropertyId` are derived from their names, so no SQL needs to be written. The other ten repositories follow the same style, for example `UserRepository.findByIdByEmail`, `TenantDueRepository.findByIdByTenantIdOrTenantEmail` and `PaymentRepository.findByIdByEscrowStatusAndReleaseEligibleDateBefore`.

5.4 Service Layer and Business Logic

The service layer contains the business rules. The property service, for example, builds a new listing with the Builder pattern and forces it into the verification queue:

```

@Override
public Property createProperty(PropertyDto.CreatePropertyRequest req) {
    Property property = Property.builder()
        .ownerId(req.getOwnerId())
        .title(req.getTitle())
        .location(req.getLocation())
        .propertyType(req.getPropertyType())
        .purpose(req.getPurpose())
        .price(req.getPrice())
        // ... area, bedrooms, bathrooms, amenities, description, images
        .status("PENDING")
        .verificationStatus("PENDING_VERIFICATION")
        .availabilityStatus("PENDING_VERIFICATION")
        .submissionDate(LocalDate.now())
        .build();
    return propertyRepository.save(property);
}

```

The most important rule is the lease approval. It changes several entities, so the whole method is wrapped in one `@Transactional` boundary: if any step fails, every change is rolled back. (The listing below is slightly shortened.)

```
@Override
@Transactional
public PropertyRequest approveRequest(String idOrRequestId) {
    PropertyRequest req = findRequestByIdOrRequestId(idOrRequestId);
    req.setStatus("APPROVED");
    PropertyRequest savedReq = requestRepository.save(req);

    String reqType = req.getRequestType() != null
        ? req.getRequestType().toUpperCase() : "";
    if (reqType.contains("RENT")) {
        Property property = propertyRepository
            .findByPropertyId(req.getPropertyId()).orElse(null);
        Double rentAmount = (property != null && property.getPrice() != null)
            ? property.getPrice() : 45000.0;
        Double utilityCharge = 5000.0;
        Double totalBilled = rentAmount + utilityCharge;
        // ... dueMonth and dueDate (25th of the current month) are computed

        TenantDue due = TenantDue.builder()
            .dueId("DUE-" + (1000 + (System.currentTimeMillis() % 90000)))
            .tenantId(req.getTenantId())
            .propertyId(req.getPropertyId())
            .rentAmount(rentAmount)
            .utilityCharge(utilityCharge)
            .totalBilled(totalBilled)
            .dueAmount(totalBilled)
            .status("UNPAID")
            .build();
        dueRepository.save(due);

        if (property != null) {
            property.setAvailabilityStatus("RENTED");
            propertyRepository.save(property);
        }
    }
    return savedReq;
}
```

Why the entity is loaded before it is changed: `findRequestByIdOrRequestId` first loads the existing `PropertyRequest`. Because the entity is then managed by the persistence context, `save()` produces an SQL UPDATE instead of an accidental INSERT, and a missing record is reported with a clear error message before any state is modified. The request type is compared with a case-insensitive `contains("RENT")` check so that both `RENT` and `RENT_REQUEST` values sent by different pages are handled.

The payment service implements the intermediary commission. For every completed rent payment it calculates the 20% platform fee and the 80% owner share, and stores them in a new settlement record:

```

Double amount = dto.getAmount() != null ? dto.getAmount() : 0.0;
Double platformFee = amount * 0.20;
Double ownerNetShare = amount - platformFee;

OwnerSettlement settlement = OwnerSettlement.builder()
    .settlementId("SET-" + (1000 + (System.currentTimeMillis() % 90000)))
    .ownerId(ownerId)
    .propertyId(dto.getPropertyId())
    .amount(amount)
    .entitledAmount(ownerNetShare)
    .commissionFee(platformFee)
    .remaining(ownerNetShare)
    .status("AVAILABLE_FOR_WITHDRAWAL")
    .type("RENTAL_INCOME")
    .build();
settlementRepository.save(settlement);

```

Escrow payments are released by a scheduled task that runs at the start of every hour. It finds all payments that are still `HELD_IN_ESCROW` and whose release date has passed:

```

@Scheduled(cron = "0 0 * * * *")
@Transactional
public void processMaturedEscrowPayments() {
    List<Payment> maturedPayments = paymentRepository
        .findByEscrowStatusAndReleaseEligibleDateBefore(
            Payment.EscrowStatus.HELD_IN_ESCROW, LocalDateTime.now());
    for (Payment payment : maturedPayments) {
        executeFundRelease(payment);
    }
}

```

5.5 REST Controller Layer

Controllers only translate HTTP into method calls: they bind parameters, trigger validation, call the service interface and convert the result (or an exception) into a `ResponseEntity` with the proper status code.

```

@CrossOrigin(originPatterns = "*", allowedHeaders = "*", allowCredentials = "true")
@RestController
@RequestMapping("/api/properties")
@RequiredArgsConstructor
public class PropertyController {

    private final PropertyService propertyService;

    @GetMapping
    public ResponseEntity<ApiResponse<List<Property>>> getProperties(
        @RequestParam(defaultValue = "ALL") String location,
        @RequestParam(defaultValue = "ALL") String propertyType,
        @RequestParam(defaultValue = "ALL") String purpose,
        @RequestParam(required = false) Double maxPrice,
        @RequestParam(required = false) Integer bedrooms) {
        List<Property> list = propertyService.searchProperties(
            location, propertyType, purpose, maxPrice, bedrooms);
        return ResponseEntity.ok(ApiResponse.ok(list.size(), list));
    }

    @PostMapping
    public ResponseEntity<ApiResponse<Property>> createProperty(
        @Valid @RequestBody PropertyDto.CreatePropertyRequest request) {
        try {
            Property created = propertyService.createProperty(request);

```

```

        return ResponseEntity.status(HttpStatus.CREATED).body(ApiResponse.ok(
            "Property submitted to TANSU Admin for verification.", created));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(ApiResponse.error(e.getMessage()));
    }
}

@PutMapping("/{propertyId}/approve")
public ResponseEntity<ApiResponse<Property>> approveProperty(
    @PathVariable String propertyId) { /* admin approval */ }
}

```

The authentication controller shows how the custom exception hierarchy is mapped to HTTP status codes:

```

@PostMapping("/signin")
public ResponseEntity<ApiResponse<AuthDto.UserResponse>> signIn(
    @Valid @RequestBody AuthDto.SignInRequest request) {
    try {
        AuthDto.UserResponse user = authService.signIn(request);
        return ResponseEntity.ok(ApiResponse.ok("Sign in successful.", user));
    } catch (UserNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ApiResponse.error(e.getMessage()));
    } catch (InvalidCredentialsException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(ApiResponse.error(e.getMessage()));
    }
}

```

HTTP status codes used by the API:

- 200 OK: successful read, update, approval, payment or deletion.
- 201 Created: a new user account or a new property listing was created.
- 400 Bad Request: DTO validation failure, or a business error such as a property or request ID that does not exist.
- 401 Unauthorized: wrong password during sign-in.
- 404 Not Found: no account exists for the e-mail used in sign-in.
- 409 Conflict: the e-mail address is already registered.

5.6 DTOs and Input Validation

Request bodies are mapped to DTO classes that carry the validation rules. Entities are never exposed directly as request payloads for creation, so the API contract can evolve independently of the database model.

```

@Data
public static class CreatePropertyRequest {
    @NotBlank(message = "Owner ID is required")
    private String ownerId;

    @NotBlank(message = "Title is required")
    private String title;

    @NotBlank(message = "Location is required")
    private String location;

    @NotBlank(message = "Purpose (RENT or SALE) is required")
    private String purpose;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than zero")
    private Double price;

    private Integer area;
    private Integer bedrooms;
    private List<String> amenities;
    // ... propertyType, bathrooms, description, images
}

```

6. REST API Endpoints

The six controllers expose more than 50 endpoint mappings (several actions are available under more than one URL for front-end convenience). The core endpoints are listed below.

Method	URL	Purpose	Request Body	Success	Error
POST	/api/auth/register (alias /signup)	Register a new tenant or owner	SignUpRequest JSON	201 Created	400 Bad Request, 409 Conflict
POST	/api/auth/signin	Authenticate a user	SignInRequest JSON	200 OK	401 Unauthorized, 404 Not Found
GET	/api/properties	Search verified listings (filters: location, propertyType, purpose, maxPrice, bedrooms)	None	200 OK	-
GET	/api/properties/{propertyId}	Get one property	None	200 OK	400 (not found)
POST	/api/properties	Owner submits a listing (status PENDING)	CreatePropertyRequest JSON	201 Created	400 Bad Request
PUT	/api/properties/{propertyId}	Update listing details	UpdatePropertyRequest JSON	200 OK	400 Bad Request
DELETE	/api/properties/{propertyId}	Delete a listing	None	200 OK	400 Bad Request
PUT	/api/properties/{propertyId}/approve	Admin approves a listing (VERIFIED)	None	200 OK	400 Bad Request
PUT	/api/properties/{propertyId}/reject	Admin rejects a listing	None	200 OK	400 Bad Request
POST	/api/requests	Submit a rent, buy or visit request	PropertyRequestDto JSON	200 OK	400 Bad Request
GET	/api/requests	List requests (optional userId / email filter)	None	200 OK	-

Method	URL	Purpose	Request Body	Success	Error
PUT	/api/requests/{id}/approve	Approve request, generate rent due, mark RENTED	None	200 OK	400 Bad Request
PUT	/api/requests/{id}/reject	Reject a request	None	200 OK	400 Bad Request
GET	/api/tenant/dues	Tenant outstanding and paid dues	None	200 OK	-
POST	/api/tenant/payments	Pay a due; creates payment, collection and settlement	PaymentDto JSON	200 OK	400 Bad Request
POST	/api/tenant/support	Create a support ticket	SupportTicketDto JSON	200 OK	400 Bad Request
GET	/api/owner/settlements	Owner earnings and settlement list	None	200 OK	-
POST	/api/owner/withdraw	Owner submits a withdrawal request	ownerId, amount, payoutMethod	200 OK	400 Bad Request
POST	/api/admin/settlements/approve	Admin approves an owner disbursement	{"settlementId": "..."}	200 OK	400 Bad Request
GET	/api/admin/finance/metrics	Platform financial KPIs	None	200 OK	-

Additional administrative endpoints:

Group	Endpoints
Listing verification	GET /api/admin/properties/pending; PUT /api/admin/properties/{id}/approve and /reject; PUT /api/admin/listings/{id}/verify
Request management	GET /api/admin/requests; PUT /api/admin/requests/{id} (status and escort officer); PUT /api/admin/requests/{id}/approve and /reject
Finance	GET /api/admin/dues; GET /api/admin/collections; GET and POST /api/admin/expenses; GET and POST /api/admin/notices; GET /api/admin/settlements
Escrow	GET /api/admin/finance/escrow; POST /api/admin/finance/release/{paymentId}
Support	GET /api/admin/support; PUT /api/admin/support/{supportId}/status
Tenant / owner convenience	GET /api/tenant/requests; GET /api/tenant/payments; GET /api/tenant/support; GET and POST /api/owner/properties

Sample JSON Requests and Responses

Sample Request: POST /api/properties

```
{
  "ownerId": "OWN-501",
  "title": "Modern 2-Bedroom Flat in Uttara Sector 7",
  "location": "Uttara",
  "propertyType": "FAMILY_HOUSE",
  "purpose": "RENT",
  "price": 32000,
  "area": 1200,
  "bedrooms": 2,
  "bathrooms": 2,
  "amenities": ["Lift", "Generator Backup", "Gas Supply"],
  "description": "Well-ventilated flat close to Uttara Metro Station."
}
```

Sample Response (201 Created):

```
{
  "success": true,
  "message": "Property submitted to TANSU Admin for verification.",
  "count": null,
  "data": {
    "id": 4,
    "propertyId": "PROP-4821",
    "ownerId": "OWN-501",
    "title": "Modern 2-Bedroom Flat in Uttara Sector 7",
    "location": "Uttara",
    "propertyType": "FAMILY_HOUSE",
    "purpose": "RENT",
    "price": 32000.0,
    "status": "PENDING",
    "verificationStatus": "PENDING_VERIFICATION",
    "availabilityStatus": "PENDING_VERIFICATION"
  }
}
```

Sample Request: POST /api/tenant/payments

```
{
  "propertyId": "PROP-GLS-01",
  "tenantId": "TNT-101",
  "amount": 100000,
  "paymentType": "RENT",
  "paymentMethod": "bKash",
  "dueId": "DUE-1042"
}
```

For this payment the service marks the due as PAID, records a collection entry and creates an owner settlement with a platform fee of BDT 20,000 and an owner share of BDT 80,000, both calculated automatically from the 80/20 rule.

Sample Error Response: POST /api/auth/register with an existing e-mail (409 Conflict)

```
{
  "success": false,
  "message": "Email already registered.",
  "count": null,
  "data": null
}
```

Note: identifiers such as PROP-4821 and DUE-1042 are generated at run time, so the values in the samples above are illustrative.

Idempotency in REST APIs

An API endpoint is idempotent if making the same request several times leaves the server in the same state as making it once. In this implementation, `PUT /api/properties/{propertyId}` is idempotent because replacing a listing with the same body five times results in the identical database state as doing it once; the same holds for `PUT /api/properties/{propertyId}/approve`, which always sets the same status values. Conversely, `POST /api/requests` and `POST /api/tenant/payments` are non-idempotent: each call creates a new request or a new payment record with a distinct identifier.

7. Database Configuration

The system uses MySQL (from a local XAMPP installation) with Hibernate ORM, configured in `application.properties`. The database `tansu_niraloy_db` is created automatically through the `createDatabaseIfNotExist=true` URL parameter, and Hibernate creates or updates the tables from the entity classes.

```
# application.properties
server.port=8080
spring.application.name=tansu-niraloy-backend

spring.datasource.url=jdbc:mysql://localhost:3306/tansu_niraloy_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true&createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=****
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

spring.jackson.date-format=yyyy-MM-dd HH:mm:ss
spring.jackson.time-zone=Asia/Dhaka
```

The database password is intentionally masked in this report. With `ddl-auto=update`, Hibernate compares the entity classes with the live schema at start-up and issues `CREATE TABLE` or `ALTER TABLE` statements for anything that is missing, while keeping the existing rows.

Setting	Development Choice	Production Recommendation
Database	MySQL local instance (XAMPP)	Managed relational database (AWS RDS / GCP Cloud SQL)
ddl-auto	update	validate or none with Flyway / Liquibase versioned migrations
Schema Changes	Automatic entity mapping creation	Explicit SQL script migrations
Credentials	Plain text in local configuration	Environment variables / secret manager injection

Setting	Development Choice	Production Recommendation
SQL Logging	show-sql=true to inspect generated queries	false, to avoid noise and sensitive data in logs

Database Tables (Entities)

Entity	Table	Purpose
User	users	Accounts with name, unique e-mail, phone and role (TENANT, OWNER, ADMIN)
Property	properties (+ property_amenities, property_images)	Listings and their verification and availability state
PropertyRequest	property_requests	Rent, buy and visit requests with status and escort officer
TenantDue	tenant_dues	Monthly rent and utility bills, overdue days and notice flags
Payment	payments	Completed payments, including escrow status and release date
TenantCollection	tenant_collections	Ledger of cleared tenant inflows
OwnerSettlement	owner_settlements	Owner entitlement (80%), platform commission (20%), disbursed and remaining amounts
OperatingExpense	operating_expenses	Platform expense vouchers recorded by admin
SupportTicket	support_tickets	Maintenance, legal-dispute and payment queries from tenants
RecoveryNotice	recovery_notices	Overdue payment notices sent to tenants
UserLoginLog	user_logins	Audit trail of successful and failed sign-in attempts

Logical Relationships

Entities are linked through business keys (for example `ownerId`, `tenantId`, `propertyId`, `dueId`) rather than JPA association annotations, which keeps each table independent and simple to query. The logical relationships are:

```
users (OWNER) --ownerId--> properties --propertyId--> property_requests
users (TENANT) --tenantId--> property_requests, tenant_dues, payments, support_tickets
property_requests (approved RENT) -- creates --> tenant_dues
tenant_dues -- paid through --> payments -- logged in --> tenant_collections
payments -- credits 80% to owner --> owner_settlements
```

Seed data: on the first start, `DatabaseSeeder` (a `CommandLineRunner`) inserts three demo accounts (one admin, one owner and one tenant) and three verified sample properties. The files `seed_properties.sql` and `properties_seed.json` provide 35 additional verified sample properties across the main residential zones of Dhaka for catalog testing.

phpMyAdmin

Server: 127.0.0.1 » Database: tansu_niraloy_db

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ operating_expenses	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ owner_settlements	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ payments	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ properties	★ Browse Structure Search Insert Empty Drop	37	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ property_amenities	★ Browse Structure Search Insert Empty Drop	197	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ property_images	★ Browse Structure Search Insert Empty Drop	71	InnoDB	utf8mb4_general_ci	2.0 MiB	-
☐ property_requests	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ recovery_notices	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ support_tickets	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ tenant_collections	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ tenant_dues	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ user_logins	★ Browse Structure Search Insert Empty Drop	124	InnoDB	utf8mb4_general_ci	16.0 KiB	-
13 tables	Sum	462	InnoDB	utf8mb4_general_ci	2.4 MiB	0 B

☐ Check all

Figure 2: MySQL database tables created automatically by Hibernate.

8. Testing and Results

The REST API was tested with Postman against the running backend on <http://localhost:8080>, using the seeded accounts and newly created data. Complete end-to-end journeys (listing, verification, rent request, lease approval, payment and settlement) were also exercised through the web client.

TC ID	Method + URL	Input / Condition	Expected Result	Actual Result	Status
TC-01	POST /api/auth/register	Valid tenant sign-up JSON	201 Created + user JSON with new userId	201 Created, userId generated (TNT-xxxx)	PASS
TC-02	POST /api/auth/register	E-mail already registered	409 Conflict	409 Conflict, "Email already registered."	PASS
TC-03	POST /api/auth/signin	Valid e-mail and password	200 OK + user details	200 OK, "Sign in successful."	PASS
TC-04	POST /api/auth/signin	Wrong password	401 Unauthorized	401 Unauthorized, "Invalid credentials."	PASS
TC-05	POST /api/properties	Valid owner listing	201 Created, status PENDING	201 Created, status PENDING	PASS
TC-06	POST /api/properties	Missing title and price <= 0	400 Bad Request (validation)	400 Bad Request	PASS
TC-07	GET /api/properties?location=Gulshan&purpose=RENT	Filtered catalog search	200 OK + matching verified listings	200 OK + filtered list with count	PASS
TC-08	GET /api/properties/PROP-GLS-01	Existing property ID	200 OK + property object	200 OK, title matched	PASS
TC-09	GET /api/properties/PROP-00000	Non-existent ID	Error with "not found" message	400 Bad Request, "Property ... not found."	PASS
TC-10	PUT /api/properties/{id}/approve	Admin approves a pending listing	200 OK, VERIFIED, AVAILABLE_FOR_RENT	200 OK, status fields updated	PASS
TC-11	POST /api/requests	Tenant rent request	200 OK, status new, requestId	200 OK, requestId REQ-xxxxx	PASS
TC-12	PUT /api/requests/{id}/approve	Admin approves rent request	200 OK, due created, property RENTED	200 OK, due UNPAID created, RENTED	PASS
TC-13	POST /api/tenant/payments	Pay the generated due	200 OK, due PAID, 80/20 settlement	200 OK, due PAID, settlement created	PASS
TC-14	POST /api/admin/settlements/approve	Valid settlementId	200 OK, status SETTLED	200 OK, remaining = 0	PASS
TC-15	DELETE /api/properties/{id}	Existing property ID	200 OK, "Property deleted."	200 OK, record removed	PASS

Postman API Test Screenshots

Screenshots of the Postman requests and responses for each test case are included below as visual evidence.

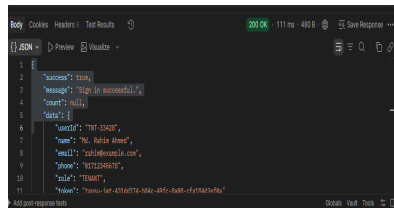


Figure 3: TC-01: POST /api/auth/register: 201 Created

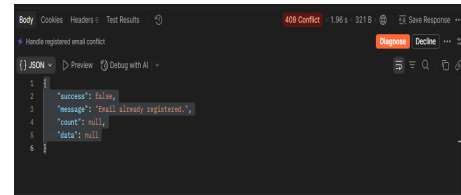


Figure 4: TC-02: POST /api/auth/register: 409 Conflict



Figure 5: TC-03: POST /api/auth/signin: 200 OK

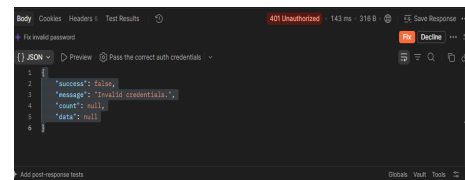


Figure 6: TC-04: POST /api/auth/signin: 401 Unauthorized

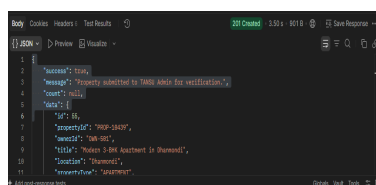


Figure 7: TC-05: POST /api/properties: 201 Created

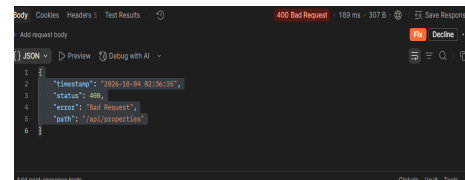
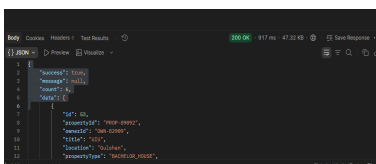
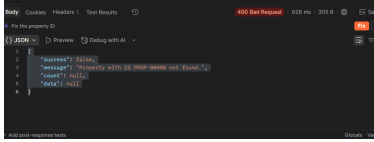


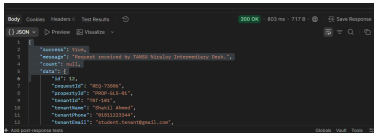
Figure 8: TC-06: POST /api/properties: 400 Bad Request



**Figure 9: TC-07: GET
/api/properties (filters): 200 OK**

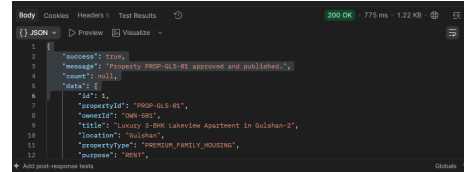


**Figure 11: TC-09: GET
/api/properties/PROP-00000:
error response**

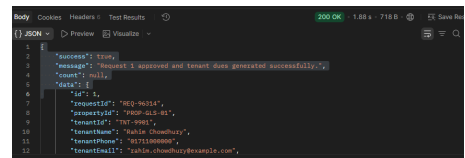


**Figure 13: TC-11: POST
/api/requests: 200 OK**

**Figure 10: TC-08: GET
/api/properties/PROP-GLS-01: 200 OK**



**Figure 12: TC-10: PUT
/api/properties/{id}/approve: 200 OK**



**Figure 14: TC-12: PUT
/api/requests/{id}/approve: 200 OK**

9. Challenges and Solutions

Challenge	Cause	How I Solved It
CORS policy blocking front-end calls	The browser blocks requests from the static front-end (Live Server, another port or file origin) to the Spring Boot server on port 8080.	Added a global <code>CorsConfig</code> class implementing <code>WebMvcConfigurer</code> that allows all origin patterns, the required HTTP methods, headers and credentials, and also added <code>@CrossOrigin</code> on the controllers.
Inconsistent request-type values	Different pages sent <code>RENT</code> , <code>RENT_REQUEST</code> or <code>RENTAL</code> , so a strict equality check missed some rent requests and no rent due was generated.	Normalized the value with <code>toUpperCase()</code> and used a <code>contains("RENT")</code> check in <code>approveRequest</code> , so all variants trigger the same billing logic.
Keeping several entities consistent on approval and payment	Approving a lease changes the request, the property and the tenant due; paying a due changes the payment, due, collection and settlement tables.	Wrapped each workflow in a single <code>@Transactional</code> service method so that either all changes are saved or all are rolled back.
Three overlapping status fields on a property	<code>status</code> , <code>verificationStatus</code> and <code>availabilityStatus</code> could become inconsistent when set by different code paths.	Centralized the rules: the entity callbacks <code>@PrePersist</code> and <code>@PostLoad</code> derive missing values, and <code>updateProperty</code> keeps <code>status</code> synchronized with <code>verificationStatus</code> .
Tenants identified by user ID in some places and by e-mail in others	The sign-in session stored the e-mail while records stored the generated <code>userId</code> , so dues and requests were not found.	Added repository methods such as <code>findByTenantIdOrTenantEmail</code> and made the services resolve an identifier through <code>UserRepository</code> before querying.
Unique business IDs	Time-based identifiers such as <code>TNT-xxxx</code> could collide for two users registered at nearly the same time.	Declared the ID columns <code>unique = true</code> and added a regeneration loop in <code>AuthServiceImpl.signUp</code> that checks <code>findByIdUserId</code> before saving.

10. Project Pages and Role Dashboard Screenshots

The Vanilla JavaScript client provides a public landing page and catalog, a combined sign-in and sign-up page, and three role-based portals. The pages communicate with the backend through the central API client in `js/api.js`. The screenshots below show the main screens of the running system.

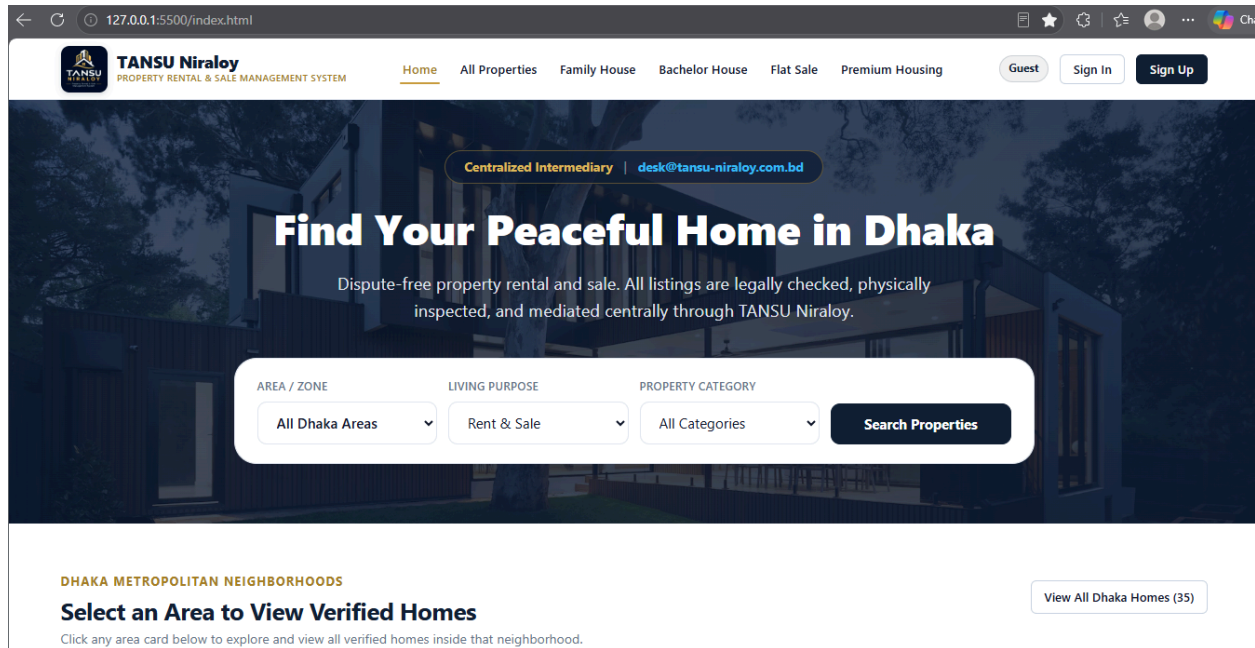


Figure 18: Landing page

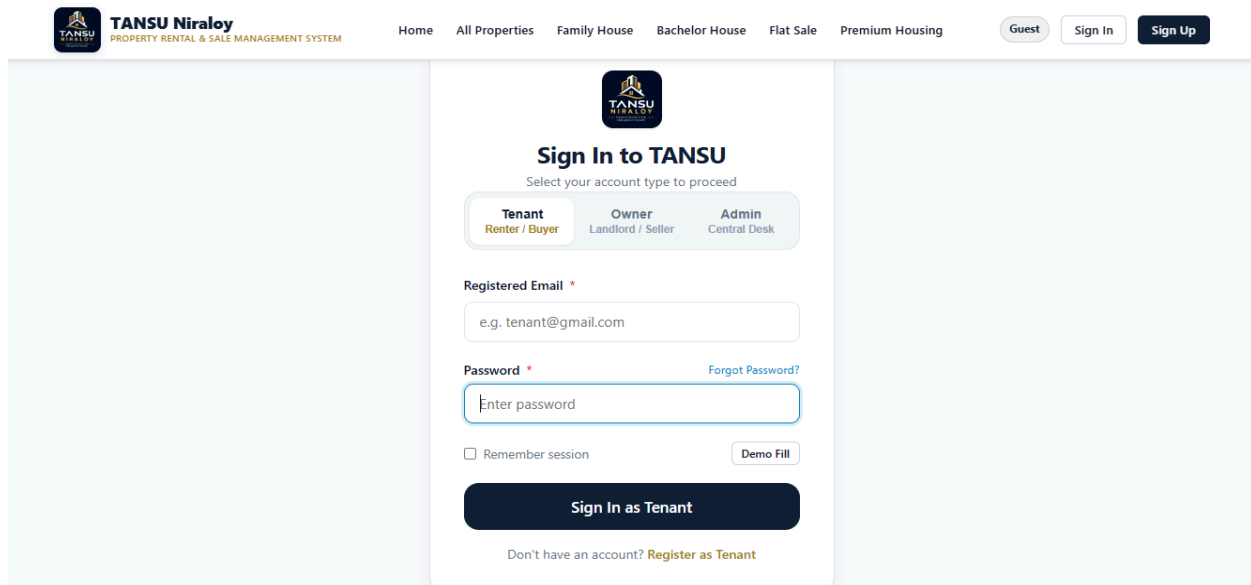


Figure 19: Sign in / Sign up page

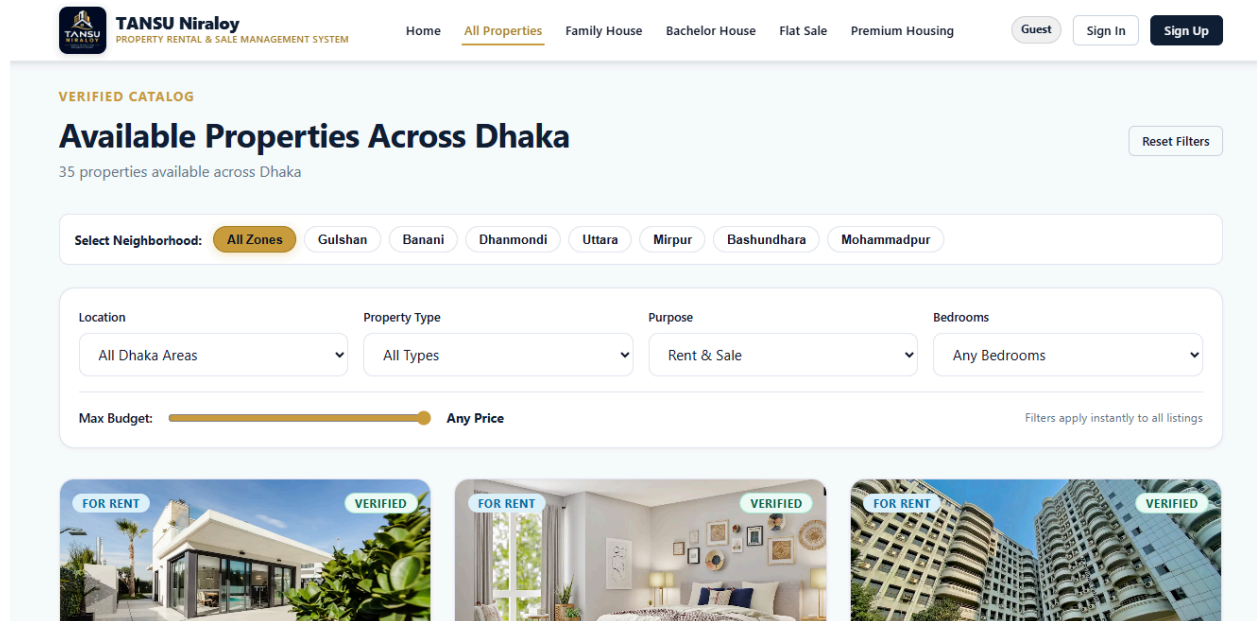


Figure 20: Property catalog with filters

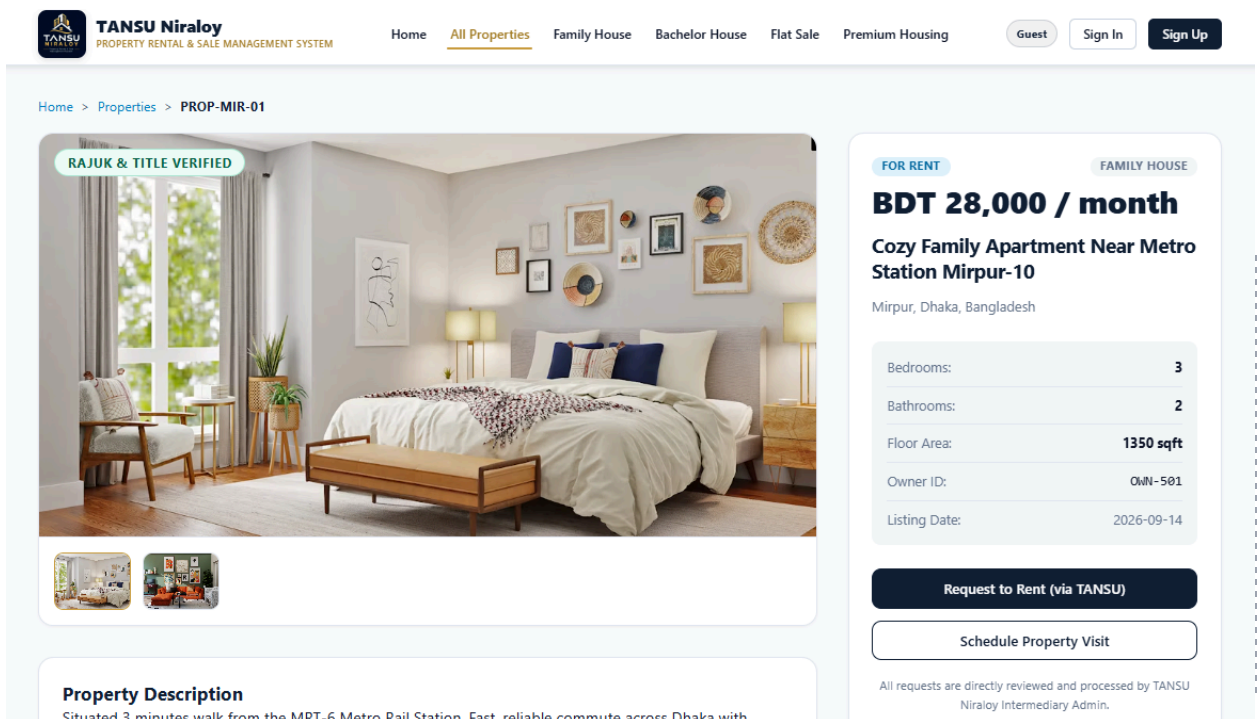


Figure 22: Tenant portal (dashboard)

[← Back to Dashboard](#)
TXN: TXN-434467


TANSU Niraloy
INTERMEDIARY CHECKOUT

YOU ARE PAYING
BDT 33,150

Subtotal

BDT 33,000

Intermediary Gateway Fee

BDT 150

Discount / Promo

- BDT 0

Total Payable
BDT 33,150

Billing Breakdown
Property ID: PROP-MIR-01
Type: RENT
Billed To: Fajjan Raiyan (TNT-62677)

Welcome Back
Select your preferred payment channel for verified clearance

256-BIT SSL

Credit / Debit Card

Mobile Banking

Net Banking

EMI Option

Card Number *
1234 5678 9012 3456

Expiration (MM/YY) *

CVV / CVC *

MM/YY

Cardholder Name *
Full name as printed on card

☐ Remember this card for future monthly rent billing

Enter Promo Code (e.g. TANSU500)

Apply

Pay BDT 33,150

256-Bit SSL Bank Grade Encryption
 PCI-DSS Level 1 Compliant
 Bangladesh Bank Clearing

[Terms of Service](#)
[Support & FAQ](#)

Secured by TANSU Gateway

Figure 23: Payment checkout page


TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

Home
All Properties
Family House
Bachelor House
Flat Sale
Premium Housing

N OWNER

AUTHENTICATED LANDLORD PORTAL

Owner Portal: Nurul Islam Dipu
Owner ID: OWN-54013 • Verified intermediary settlement protocol

TOTAL PROPERTIES
0
Registered under Owner ID

VERIFIED LISTINGS
0
Active in Public Catalog

TOTAL EARNINGS
BDT 22,800
From Rented & Sold Flats

AVAILABLE BALANCE
BDT 0
Request Withdrawal

TOTAL WITHDRAWN
BDT 22,800
Disbursed to Your Account

My Properties

Rented/Sold Earnings Ledger

Withdrawal Requests & Payouts

My Listed Properties
Only properties matching your ownerId are displayed. Administrative verification is required before any listing goes live.

Property ID
Title & Location
Purpose
Asking Price
Verification Status
Availability
Actions

You haven't listed any properties yet.
+ Add Your First Listing

Figure 24: Owner portal (dashboard)



TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

[Home](#) [All Properties](#) [Family House](#) [Bachelor House](#) [Flat Sale](#) [Premium Housing](#)

N

OWNER

OWNER SUBMISSION PROTOCOL

Submit Property for Verification

← Back to Dashboard

All listings undergo mandatory administrative inspection before public catalog inclusion.

Property Title *

e.g. Modern South-Facing 3-BHK Apartment with Balcony

Location (Dhaka) *

Gulshan

Property Classification *

Family House

Listing Purpose *

For Rent

Asking Price (BDT) *

e.g. 45000

Area (sqft) *

e.g. 1850

Bedrooms *

3

Bathrooms *

3

Verified Amenities Available

☒ Elevator

☒ Generator Backup

☒ Car Parking

☒ Security Guard

☐ CCTV Surveillance

☐ Gas Line

☒ Balcony

☐ Fitness Center

Detailed Description & House Rules *

State floor level, orientation (e.g. south-facing), nearby landmarks, utility bills inclusion, or family preferences...

Figure 25: Owner add property page



TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

[Home](#) [All Properties](#) [Family House](#) [Bachelor House](#) [Flat Sale](#) [Premium Housing](#)

T

ADMIN

CENTRAL INTERMEDIARY CONTROL

TANSU Administration Console

INTERMEDIARY DESK ACTIVE

Platform-wide verification, escrow clearing, and dispute-free intermediary management

INTERMEDIARY OPERATIONS

Central Dashboard

Verify Listings 1

Verified Properties

Rental Requests 1

Purchase Requests

Visit Requests

Finance & Settlements

Transactions Ledger

Complaints & Support

Reports & Vetting

System Settings

TOTAL PROPERTIES

36

Across Dhaka zones

VERIFIED LISTINGS

35

Active in catalog

PENDING LISTINGS

1

Needs inspection

ACTIVE RENTALS

28

Central lease contracts

Intermediary Financial Health Summary

Official Audit Ledger

TOTAL PAYMENTS CLEARED

BDT 74,500

Automated receipts issued

OWNER SETTLEMENTS DISBURSED

BDT 22,800

Direct BEFTN / NPSB

PENDING SETTLEMENTS

BDT 36,000

Awaiting audit clearance

SYSTEM OPERATIONS FEE

BDT 2,500

Platform maintenance

Urgent Action Queues

[Review Owner Submissions →](#)
[Assign Field Officers to Escorted Visits →](#)
[Process Formal Lease Drafting →](#)

Intermediary Architecture Rule

Ensure field officers never release property keys until lease contracts are executed and first-month rental settlement has entered the clearing account.

Figure 26: Admin portal (dashboard)



TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

[Home](#)
[All Properties](#)
[Family House](#)
[Bachelor House](#)
[Flat Sale](#)
[Premium Housing](#)

T

ADMIN

LISTING VETTING QUEUE

Owner Listing Verification

[← Back to Dashboard](#)

Validate RAJUK approval, ownership deeds, and realistic pricing before approving to public catalog.

Mandatory Rule: Property owners are strictly prohibited from verifying their own listings. Every item below was submitted by an external owner and is currently hidden from public search until verified.

Property ID	Owner ID	Title & Specifications	Purpose	Price	Submission Date	Administrative Decision
PROP-10439	OWN-501	Modern 3-BHK Apartment in Dhanmondi Dhanmondi • 3 Beds, 3 Baths, 1450 sqft	RENT	BDT 35,000	2026-10-04	View Details Approve Reject



TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

Centralized Property Rental & Property Sale Management Platform. Mediating transparent, legally vetted, and secure property deals across Dhaka, Bangladesh.

Property Categories

Family Houses

Bachelor Accommodations

Portals & Governance

Tenant Portal

Owner Portal

Intermediary Guarantee

Owner and Tenant never transact directly. Every listing undergoes mandatory verification, visit scheduling is escorted, and payments are securely held in escrow.

Figure 27: Admin listing verification queue



TANSU Niraloy
PROPERTY RENTAL & SALE MANAGEMENT SYSTEM

[Home](#)
[All Properties](#)
[Family House](#)
[Bachelor House](#)
[Flat Sale](#)
[Premium Housing](#)

T

ADMIN

FINANCIAL CONTROL & SETTLEMENT SYSTEM

Central Clearing & Settlements

[Transactions Ledger →](#)
[← Dashboard](#)

Comprehensive intermediary accounting: Tenant Inflows, Outstanding Arrears & Recovery Notices, Landlord Remittances, and Operating Expenses.

TENANT COLLECTIONS INFLOW

BDT 74,500

Verified rent & deposit receipts

TENANT OUTSTANDING DUES

BDT 133,000

0 Overdue Accounts

OWNER SETTLEMENTS DISBURSED

BDT 22,800

Disbursed to landlord accounts

OPERATING EXPENSES

BDT 2,500

Field inspections & IT cloud

Tenant Rent Collections

Tenant Outstanding Dues & Notices

Owner Settlements Pipeline

Operating Expenses

Tenant Rent Collections & Inflow Ledger

Real-time record of all rental payments collected from tenants, detailing which flat was rented, billing period, and payment gateway channel.

Collection / Trx ID	Tenant Details	Rented Flat & Location	Type & Period	Collected Amount	Payment Channel	Date & Time	Action
---------------------	----------------	------------------------	---------------	------------------	-----------------	-------------	--------

Figure 28: Admin finance and settlements

11. Conclusion and Future Work

The TANSU Niralogy Spring Boot backend was successfully engineered to solve the trust and transparency problems of the urban rental and property-sale market. All key objectives were met: a layered REST API with property, request, billing, payment, settlement and support workflows; relational persistence with Spring Data JPA and MySQL; Bean Validation of all inputs; automatic rent-due generation and an 80/20 settlement and escrow engine; and a clean object-oriented design built on service interfaces, dependency injection, the Builder, Static Factory, DTO and Repository patterns. The behaviour of the system was validated through API testing and complete multi-role workflows in the web client.

Future Work:

- Harden security with Spring Security, BCrypt password hashing and signed JWT tokens, including role-based authorization on the `/api/owner` and `/api/admin` routes.
- Add global exception handling with `@ControllerAdvice` and `ProblemDetail` so that missing resources consistently return `404 Not Found` and access violations return `403 Forbidden`.
- Make the approval and payment endpoints strictly idempotent by checking the current status before creating dues or settlements.
- Add automated unit and integration tests using JUnit 5, Mockito and MockMvc.
- Replace string-based references between entities with JPA relationships (`@ManyToOne`, `@OneToMany`) and add pagination to list endpoints.
- Integrate a real payment gateway sandbox (for example SSLCommerz or the bKash merchant API) and scheduled monthly due generation.
- Move database credentials to environment variables and use Flyway migrations for production deployment.

References

- [1] Spring, "Spring Boot Reference Documentation." [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/> [Accessed: Oct. 2026].
- [2] Spring, "Spring Data JPA Reference Documentation." [Online]. Available: <https://docs.spring.io/spring-data/jpa/reference/> [Accessed: Oct. 2026].
- [3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.
- [4] Oracle, "Java Platform, Standard Edition Documentation." [Online]. Available: <https://docs.oracle.com/en/java/> [Accessed: Oct. 2026].
- [5] Hibernate, "Hibernate ORM User Guide." [Online]. Available: <https://hibernate.org/orm/documentation/> [Accessed: Oct. 2026].
- [6] Oracle, "MySQL Reference Manual." [Online]. Available: <https://dev.mysql.com/doc/> [Accessed: Oct. 2026].

